

Architectural Blueprint and Frontend Build Plan for Offline-First Edge-Computed Geospatial Applications

The conceptualization of edge-compiled Geographic Information Systems (GIS) for mobile applications represents a profound paradigm shift from traditional cloud-centric infrastructure.¹ The architecture detailed in this document defines the engineering specifications for the "FreeHike" application, a system that shifts the spatial Extract, Transform, Load (ETL) pipeline entirely to the edge device. By allowing raw OpenStreetMap (OSM) Protocolbuffer Binary Format (.pbf) files to be compiled natively into PMTiles archives directly on consumer hardware, the architecture eliminates the continuous, scaling cloud compute and storage costs associated with global Content Delivery Networks (CDNs).¹

This document provides an exhaustive architectural analysis and a definitive frontend build plan to govern the interface layer of this system. It strictly addresses the constraints of running a React and TypeScript Single Page Application (SPA) inside a Capacitor WebView. The analysis formulates solutions for establishing zero-latency bridges between MapLibre GL JS and local file systems, mitigating mobile operating system background memory limits, optimizing React rendering lifecycles for high-frequency native events, and executing completely offline map styling.¹

1. The MapLibre & PMTiles Offline Bridge Architecture

The fundamental and most critical structural challenge in deploying PMTiles within a Capacitor WebView environment is establishing a reliable, high-bandwidth, and low-latency mechanism for HTTP Byte-Range requests.¹ PMTiles is a single-file, cloud-optimized archive format designed specifically for hosting tilesets without an active tile server. It relies on querying an internal directory structure—sorted via a packed Hilbert space-filling curve—through precise HTTP Range requests to extract specific Mapbox Vector Tile (MVT) or MapLibre Tile (MLT) payloads without loading the massive archive into memory.¹

Systemic Failures of Default Capacitor Protocols

MapLibre GL JS natively expects vector tiles to be served via standard HTTP endpoints, utilizing browser fetch or XMLHttpRequest APIs. In a standard Capacitor application, local files residing in the device's application data directory are not served by a traditional remote server. Instead, they are routed through custom URL scheme handlers, specifically utilizing `capacitor://localhost/_capacitor_file_/` on iOS and `http://localhost/_capacitor_file_/` on Android.¹ Attempting to utilize this default Capacitor bridging mechanism for multi-gigabyte .pmtiles archives introduces critical, mathematically guaranteed systemic failures across both primary

mobile operating systems.

On the iOS platform, WebKit's underlying WKURLSchemeHandler exhibits documented, structural defects regarding the handling of byte-range requests.⁵ When MapLibre attempts to fetch a 16-kilobyte vector tile by appending a Range: bytes=1000-17000 header to a capacitor:// request, the WebKit scheme handler frequently drops, overrides, or ignores the header entirely.⁵ Consequently, the WebView attempts to read the entire multi-gigabyte map archive into its volatile memory space to satisfy the request. Modern iOS memory governors, specifically the memorystatus subsystem known as "Jetsam," impose strict ActiveHard resident memory limits on WebViews, frequently capped around 1.5 to 2 gigabytes.¹ Loading an entire regional PMTiles file instantly breaches this ceiling, resulting in an uncatchable SIGKILL termination and a silent application crash.¹

The Android environment presents an equally fatal, albeit distinct, architectural limitation. The Capacitor local file server processes incoming range requests by parsing the HTTP headers into a standard 32-bit Java Integer.⁸ Because the maximum representable value of a signed 32-bit integer is 2,147,483,647, requesting any byte offset beyond 2.14 gigabytes causes an immediate integer overflow exception, causing the local server thread to fail and return corrupted or empty responses to the WebView.⁸ Given that a densely mapped European region compiled into vector tiles routinely exceeds this 2.14GB boundary, the default Capacitor file server is fundamentally incompatible with the FreeHike storage requirements.

Evaluating Alternative Communication Vectors

Given the insurmountable limitations of the default Capacitor file protocol, the architecture must implement a specialized mechanism to connect MapLibre's rendering engine to the local .pmtiles file. The engineering analysis evaluates three distinct paradigms for crossing the native-to-web boundary.

Bridge Architecture	Execution Mechanism	Systemic Constraints and Limitations	Architectural Viability
Local Native HTTP Server	A dedicated Capacitor plugin initializes a lightweight local web server (e.g., utilizing standard TCP sockets on a randomized port), entirely circumventing the WebView scheme	Introduces severe battery drain due to continuous socket listening. Requires complex background execution lifecycle management to prevent the OS from suspending the server port.	Low

	handlers and serving files directly to localhost:PORT. ⁹	Triggers strict, user-hostile local network permission prompts on modern iOS versions.	
Native Chunking Bridge (JSI)	Implementing a custom JavaScript interface that invokes a Capacitor plugin (e.g., capacitor-file-chunk) to read file segments natively and return binary data via the asynchronous bridge. ¹¹	Serializing massive volumes of binary data across the asynchronous WebView bridge introduces massive overhead. Even if converted to Uint8Array, the bridging process blocks the main JavaScript thread, degrading the MapLibre panning frame rate. ¹¹	Moderate
The Origin Private File System (OPFS)	The native compiler stream-copies the final archive into the browser's OPFS sandbox. Web Workers access the file synchronously via FileSystemSyncAccessHandle, completely bypassing the Capacitor bridge during active rendering. ¹	Requires momentarily duplicating the storage footprint during the transfer phase. However, it provides unrestricted, low-latency, synchronous byte-level access directly mapped to the solid-state drive.	High

The OPFS Integration Seam

To achieve zero-latency rendering and eliminate main-thread blocking, the architecture must completely bypass the Capacitor asynchronous bridge during active map interaction. The definitive solution relies on leveraging the Origin Private File System (OPFS). OPFS is a highly

optimized, origin-sandboxed virtual file system accessible directly within the modern browser engine.¹³ It is specifically engineered for high-performance read and write access to raw binary data, executing file operations three to four times faster than standard IndexedDB implementations.¹ Crucially, unlike the traditional Cache API or IndexedDB—which require the browser's structured clone algorithm to serialize and deserialize data, leading to massive memory spikes—OPFS permits true byte-level access.¹

Because the native Rust compilation engine operates outside the restrictive WebView sandbox to maximize physical RAM usage and bypass WebAssembly constraints, it writes the final .pmtiles archive to the native operating system's application data directory.¹ Consequently, an architectural "seam" must be implemented to bridge the native storage and the browser sandbox. Upon the successful conclusion of the Rust compilation pipeline, a native Capacitor file management routine must stream the completed binary archive directly into the WebView's OPFS container.¹

Once the file resides within OPFS, the application must intercept MapLibre's internal HTTP requests and redirect them to the local file system. The @protomaps/pmtiles JavaScript library supports the injection of custom data retrieval strategies by exposing a generic Source interface.¹⁴ The implementation requires the creation of a custom class, conceptually defined as OPFSSource, which implements this interface by overriding the core getBytes(offset, length) method.¹⁵

To ensure the MapLibre rendering cycle maintains a strict 60 frames per second (FPS), this data retrieval must not execute on the main UI thread. The OPFSSource class must delegate the getBytes invocation to a dedicated background Web Worker.¹ Within this Web Worker environment, the application utilizes the FileSystemSyncAccessHandle API, a specialized OPFS interface only available in worker contexts.¹ This handle permits synchronous read() operations directly against the .pmtiles file residing on the device's NVMe or UFS flash storage.¹

The integration culminates by registering this custom source via MapLibre's protocol injection API. By executing maplibregl.addProtocol('pmtiles', customProtocol.tile) during application initialization, the rendering engine's internal WebGL tile requests are seamlessly intercepted.¹⁷

When the map attempts to load pmtiles://local-region, the request is routed through the OPFSSource, dispatched to the Web Worker, read synchronously from the flash storage, and returned as a pure zero-copy ArrayBuffer directly to the MapLibre WebGL context.¹ This architecture entirely evades the defective capacitor:// URL schemes, eliminates Cross-Origin Resource Sharing (CORS) header complexities, avoids base64-encoding memory bloat, and provides desktop-class rendering velocities on mobile edge hardware.

2. Native-to-WebView Event Bridge

The edge-computation of regional topographic maps from raw .osm.pbf files and GeoTIFF Digital Elevation Models is a severely prolonged workload.¹ Processing a planetary subset requires the native Rust engine to execute a multi-threaded pipeline—encompassing a primary node indexing pass, a secondary geometry reconstruction pass, mathematical transformation to the Web Mercator projection (EPSG:3857), Ramer-Douglas-Peucker (RDP) vector

simplification, and final Mapbox Vector Tile (MVT) serialization.¹ Depending on the selected bounding box, this compilation can sustain maximum CPU utilization for durations ranging from several minutes to over an hour.¹

Throughout this extensive duration, the native Rust core emits a continuous stream of high-frequency telemetry via Mozilla UniFFI and the Capacitor plugin bridge, detailing phase transitions, processed block counts, and exact byte offsets.¹ Relaying this dense, high-frequency data across the asynchronous boundary to the React Single Page Application introduces profound performance liabilities.

The React Reconciliation Crisis

Modern frontend frameworks like React operate on a declarative state paradigm. The standard architectural approach for handling incoming data is to dispatch the payload to a component's internal state utilizing the `useState` hook, or to a global context provider. However, if the Capacitor event listener receives compilation telemetry at a rate of fifty to one hundred events per second, mapping these events directly to React state triggers a catastrophic cascade of Virtual DOM reconciliations.²⁰

Each state mutation forces React to re-evaluate the component tree, generate a new Virtual DOM representation, execute a diffing algorithm against the previous tree, and apply the necessary mutations to the actual browser DOM. Executing this cycle dozens of times per second consumes massive CPU cycles, blocks the main JavaScript thread, induces aggressive garbage collection pauses due to the constant allocation of new state objects, and ultimately results in severe UI freezing and dropped frames.²⁰ For a mobile application simultaneously attempting to reserve maximum hardware resources for a background Rust compilation thread, wasting CPU cycles on unnecessary DOM diffing is structurally unacceptable.

The requestAnimationFrame Architecture

To ensure the Download & Compile View updates smoothly at a strict 60 FPS without triggering massive React component tree re-renders, the architecture must entirely decouple the high-frequency telemetry stream from the React rendering engine.²⁰ The prescribed solution leverages a unidirectional data flow utilizing mutable references (`useRef`) perfectly synchronized with the browser's native rendering pipeline via the `requestAnimationFrame` API.²⁰

Under this paradigm, the React application mounts a single, global event listener via the Capacitor plugin API (e.g., `CompilerPlugin.addListener('compilation_progress', handler)`) when the compilation initiates. Crucially, the handler function explicitly avoids dispatching the payload to any React state management hook. Instead, it strictly mutates a shared `useRef` object, conceptually executing `progressRef.current = payload.bytesProcessed`. Modifying the `.current` property of a React reference is a synchronous operation that intentionally does not trigger a component render cycle.²⁰ This allows the application's memory space to absorb thousands of native events per second with near-zero CPU overhead, acting merely as a high-speed data sink.

To visualize this data, the Download & Compile View component initializes a recursive

requestAnimationFrame loop upon mounting.²¹ The requestAnimationFrame API provides a timestamp parameter and is inherently synchronized with the device's display refresh rate, guaranteeing that the callback runs at the exact correct point in the browser's rendering pipeline before the next repaint.²² Within this recursive loop, the application reads the latest integer value from the progressRef.current object and explicitly updates the DOM node representing the progress bar. This is achieved by utilizing a secondary useRef pointing directly to the HTML element (e.g., progressBarRef.current.style.transform = scaleX(\${calculatedPercentage})).²⁰

By updating CSS variables or transform properties directly on the DOM node, the application completely bypasses the React Virtual DOM diffing engine. The Capacitor bridge absorbs the compilation data as rapidly as the operating system permits, while the UI polls the latest value strictly when the monitor is ready to draw the next frame. To prevent severe memory leaks when the component unmounts or the compilation concludes, the architecture must meticulously capture the integer ID returned by requestAnimationFrame and invoke cancelAnimationFrame within the React useEffect cleanup function.²¹

Idempotency and Background Governor Rehydration

The architecture must also account for the hostile nature of mobile background execution environments. The operating system actively combats prolonged background execution to preserve battery endurance and thermal stability.¹ On iOS, the BGProcessingTask API relies on opportunistic scheduling and enforces a strict, unnegotiable execution window of approximately 295 seconds (4.9 minutes).¹ When the kernel invokes the expiration handler, the application has only seconds to suspend operations before facing a forceful SIGKILL termination.¹ Android's environment, governed by the Low Memory Killer Daemon (LMKD) and Doze Mode, imposes similar, albeit less predictable, constraints even when utilizing Foreground Services.¹

Consequently, the native Rust compiler is architected as a deeply resilient, chunked, and idempotent state machine, persistently writing its exact execution checkpoint to an embedded redb database on the flash drive.¹ Because the OS can abruptly suspend the application, the React frontend cannot assume a continuous, unbroken event stream. The event bridge must support robust state rehydration.

When the user foregrounds the application after a suspension, the React frontend must execute a synchronous query across the Capacitor bridge (e.g., CompilerPlugin.getJobState()) to fetch the latest checkpointed spatial chunk ID, active phase, and processed byte offset.¹ The UI uses this baseline data to re-initialize the progressRef.current object and instantly resume the requestAnimationFrame polling loop, ensuring the user interface accurately reflects the compilation status regardless of how many times the operating system has killed and resurrected the background process.

3. UI/UX Architecture & State Management

The structural design of the interface requires a meticulously planned component hierarchy to isolate the heavyweight WebGL context of the map renderer from the transient,

high-frequency state of the compilation interface. The UI/UX architecture must seamlessly transition users from selecting planetary regions to tracking multi-hour background tasks, and finally to navigating high-resolution topographical maps without network connectivity.

State Management: Zustand versus React Context

For managing the global application state—which encompasses the user's active GPS coordinates, the currently selected offline region, and the status of the native compilation engine—the architecture explicitly mandates the use of Zustand over the native React Context API.²⁰

React Context is structurally unsuited for high-performance, dynamic geospatial applications. The fundamental flaw of the Context API lies in its provider-tree dependency mechanism; whenever a value within a Context Provider is mutated, React forcefully re-renders every single component subscribed to that context, regardless of whether the specific subset of data consumed by the component actually changed. In a complex map application, utilizing Context for rapidly changing data (such as GPS telemetry or bounding box coordinates) results in continuous, cascading re-renders across the entire interface.

Zustand operates fundamentally differently, existing entirely outside the React component lifecycle. It allows discrete components to subscribe to granular, isolated slices of the state tree utilizing selectors. For instance, a component can execute `const currentPhase = useCompilerStore(state => state.phase)`. If a completely unrelated value in the store updates (e.g., `state.bytesProcessed`), the `currentPhase` subscriber will definitively not re-render. Furthermore, because the Zustand store is not bound to the React context tree, it permits state modifications directly from external vanilla JavaScript files or global event listeners. This is an absolute prerequisite for hooking into the Capacitor event bridge or WebSocket streams outside of the React component hierarchy.²⁰

Component Tree and View Orchestration

The application logic is partitioned into two primary routing domains. Managing the transition between these views requires a specialized persistent mounting strategy to mitigate the severe initialization latency inherent to WebGL contexts.

The Download & Compile View

This interface governs the ingestion of raw data and the orchestration of the edge-compute workload.¹ It provides the primary user interaction point for defining the scope of the offline region.

- **Bounding Box Selector:** A lightweight spatial selection tool. Rather than initializing a second heavy MapLibre instance, this component utilizes an extremely simplified, low-fidelity canvas or an SVG-based map interface allowing the user to draw a geographic rectangle. These coordinates are captured and pushed to the native layer to initiate the targeted Overpass API or Geofabrik download.¹
- **Compilation HUD:** This component utilizes the `requestAnimationFrame` architecture defined in the previous section to display highly performant progress telemetry.²² It must provide a visual breakdown of the multi-pass Rust pipeline (e.g., Phase 1: Node Indexing,

Phase 2: Geometry Reconstruction, Phase 3: Terrain Assembly).¹

- **Thermal and Suspension State Handlers:** To provide an honest user experience regarding edge computation, the UI must explicitly reflect hardware realities. By polling the native iOS `ProcessInfo.thermalState` or Android `THERMAL_STATUS`, the Zustand store can detect if the device is entering a thermal throttling state.¹ If detected, the UI must dynamically inform the user that the compilation thread pool has been automatically reduced to prevent hardware damage, explaining the corresponding increase in compilation time. It must also provide manual Pause, Resume, and Cancel controls mapped to the Rust engine's chunked API.¹

The Interactive Map View

The Interactive Map View houses the full-screen MapLibre GL JS canvas, the core visualization engine of the FreeHike application.

Initializing a MapLibre WebGL context is computationally expensive, involving the compilation of complex fragment and vertex shaders and the allocation of GPU memory buffers.¹ In a standard SPA, navigating from the map to a settings menu or the Download View via a library like React Router would unmount the `<canvas>` element from the DOM, destroying the WebGL context. Returning to the map would trigger a complete re-initialization, severely degrading the perceived performance of the application.

The architecture mandates that the MapLibre container remains permanently mounted in the DOM hierarchy from the moment the application initializes. When the user navigates away from the map, the application utilizes Zustand state to dynamically apply CSS directives (`visibility: hidden` or `display: none`) to the MapLibre wrapper container.²² This hides the canvas from the user without destroying the underlying GPU state, enabling instantaneous, zero-latency restorations when the user switches back to the map.

- **Offline Map Switching:** When a background compilation finishes and the `.pmtiles` archive is transferred into OPFS, the user must be able to switch to the new region immediately. The Interactive Map View observes the Zustand store for changes to the `activeRegionId`. Upon mutation, it does not destroy the map; instead, it invokes MapLibre's native APIs, such as `map.setStyle(newStyle)` or accesses the existing source via `map.getSource('offline-tiles')` and invokes `setData()` or updates the URL.²⁴ This hot-swaps the underlying vector topology on the fly.
- **User Location Tracking (GPS):** Real-time GPS tracking is integrated via Capacitor's native Geolocation plugin. The plugin pushes high-frequency coordinate updates into a dedicated Zustand store. The Interactive Map View subscribes to this coordinate stream and programmatically updates a dedicated MapLibre `GeoJSONSource` representing the user's current position, heading, and accuracy radius, allowing MapLibre to re-render the user's location marker dynamically at 60 FPS without touching the React DOM.¹

4. Map Styling & Layering

The culmination of the Rust compilation pipeline is a densely packed PMTiles archive containing complex vector geometries. The Rust engine extracts foundational OpenStreetMap

primitives—nodes, ways, and relations—and translates them into specific vector layers (e.g., highway, waterway, natural, landuse).¹ To visualize these mathematical vectors, MapLibre requires a rigorously formatted style.json document that maps the raw data layers to visual rendering directives.

For an application constrained to a strict "zero-cost" and "local-first" paradigm, the style.json must be entirely autonomous. It cannot rely on external Mapbox APIs or commercial cloud endpoints for map tiles, typography, or iconography.²⁶

Structuring the Offline style.json

The JSON specification must explicitly route all resource requests to the local device storage using custom protocols and relative paths. The architecture relies on the custom protocol registered during the application initialization.²⁸

The sources block of the style.json establishes the connection to the local OPFS-backed PMTiles archive. Instead of pointing to a remote https:// endpoint, it utilizes the registered pmtiles:// scheme.²⁹

```
JSON
{
  "sources": {
    "freehike-basemap": {
      "type": "vector",
      "url": "pmtiles://local-region-data",
      "minzoom": 5,
      "maxzoom": 14
    }
  }
}
```

The layers block maps the internal source layers generated by the Rust engine to specific styles. Because the Rust engine filters irrelevant OSM data during Pass 1, the style.json only needs to define directives for features relevant to hiking, such as paths categorized by the sac_scale tag, water bodies, and generated contour isolines.¹

```
JSON
```

```

|
|
| "id" "trails-alpine"
| "type" "line"
| "source" "freehike-basemap"
| "source-layer" "highway"
| "filter" "all"
| "==" "class" "path"
| "==" "sac_scale" "alpine_hiking"
| "paint"
| "line-color" "#d9432f"
| "line-width" 2.5
| "line-dasharray"
|
|

```

Typography and Symbolism Governance

Standard web-based map styles dynamically fetch typography (glyphs) and iconography (sprites) from external HTTP endpoints.²⁷ To guarantee complete offline functionality, the FreeHike application bundle must include pre-compiled Protocolbuffer (.pbf) font stacks and sprite sheets natively within the application binary.³²

- **Offline Glyphs:** Rendering text labels on vector maps requires fonts to be converted into Signed Distance Field (SDF) .pbf ranges (e.g., 0-255.pbf). The necessary font families (such as Noto Sans) must be packaged directly into the public/assets/fonts/ directory of the React SPA during the build process.³⁴ The style.json is configured to resolve these fonts via a relative local path: "glyphs": "/assets/fonts/{fontstack}/{range}.pbf".² Alternatively, a dedicated custom protocol (e.g., glyphs://) can be registered via addProtocol to intercept MapLibre's font requests and resolve them directly from local storage.³⁵
- **Offline Sprites:** Similarly, high-resolution .png sprite sheets and their accompanying .json index files—which map icon names (e.g., campsite, peak) to specific pixel coordinates on the image—must reside locally. The style definition is mapped as "sprite": "/assets/sprites/hiking-icons".²⁷ This guarantees that even in deep wilderness without cellular reception, the map can render detailed topographical symbols.

Completely Offline Dynamic Theming (Dark/Light Mode)

Implementing dynamic Dark and Light modes poses a unique challenge in an offline environment. The naive approach involves maintaining two entirely separate style.json files and executing map.setStyle() when the user toggles the theme. However, invoking setStyle() forces MapLibre to completely tear down its existing state, destroy the WebGL vertex and fragment shaders, and recalculate the geographic topology for the entire viewport. This operation is computationally heavy and results in a jarring visual flash.

The optimized architecture achieves instantaneous, zero-latency theme transitions by

leveraging MapLibre's data-driven styling and programmatic paint property manipulation.²⁵ When the global Zustand state detects a theme change—either triggered by a manual UI toggle, the native Capacitor Preferences API, or an OS-level media query—a React useEffect hook intercepts the event. Instead of swapping the entire style document, the hook iterates through predefined thematic dictionaries and invokes the `map.setPaintProperty()` API on specific layer IDs.²⁵

By executing commands such as `map.setPaintProperty('water-layer', 'fill-color', '#1a365d')` or `map.setPaintProperty('trails-alpine', 'line-color', '#ffb3a7')`, the application exclusively alters the visual paint properties. The MapLibre rendering engine processes these commands by updating the WebGL pixel shaders immediately, without dropping or recalculating the underlying, memory-heavy geometric vector buffers. This results in a fluid, 60 FPS transition between daytime and nighttime topographies that functions flawlessly without network access and without spiking the mobile device's CPU.²⁵

Prioritized Frontend Implementation Plan

Synthesizing the architectural analysis, the following phased execution strategy provides a deterministic roadmap for the frontend engineering team to construct the FreeHike interface, guaranteeing seamless integration with the native Rust compilation engine.

Phase 1: Foundational Scaffolding & Native Control Plane (Weeks 1-2)

1. **Capacitor Plugin Instantiation:** Scaffold the iOS (Swift) and Android (Kotlin) native plugin interfaces. Establish the strict UniFFI boundaries required to invoke the Rust engine's `compile_chunk(job)` API.
2. **Zustand State Architecture:** Implement the global Zustand stores, strictly separating the user's interactive state (GPS, selected region) from the transient compilation state.
3. **The Polling Infrastructure:** Construct the `useRef` and `requestAnimationFrame` polling mechanism within the Download View. Map this mechanism to a Capacitor event listener to intercept the high-frequency native telemetry stream, ensuring the DOM updates at 60 FPS without triggering React reconciliations.

Phase 2: The OPFS Integration Seam (Weeks 3-4)

1. **Native File Mover:** Develop the native Capacitor routine responsible for stream-copying the completed `.pmtiles` archive from the OS application data directory directly into the WebView's origin-sandboxed OPFS layer to circumvent the `WKURLSchemeHandler` limitations.
2. **Custom PMTiles Source Implementation:** Author the `OPFSSource` TypeScript class implementing the `@protomaps/pmtiles Source` interface.
3. **Web Worker Delegation:** Inside the `OPFSSource`, override the `getBytes(offset, length)` method to dispatch a message to a dedicated Web Worker. Within the worker, utilize `navigator.storage.getDirectory()` and `createSyncAccessHandle()` to execute synchronous, zero-latency byte-range reads against the local `.pmtiles` payload.

Phase 3: MapLibre Rendering & Offline Configuration (Week 5)

1. **Protocol Registration:** Ensure that during SPA initialization, the application invokes `maplibregl.addProtocol('pmtiles', protocol.tile)` utilizing the newly engineered `OPFSSource`.
2. **Local Resource Bundling:** Package the necessary Protocolbuffer glyphs (`{{fontstack}}/{range}.pbf`) and high-resolution UI sprite sheets directly into the `React public/assets` directory, completely eliminating external network dependencies.
3. **Style JSON Formulation:** Construct the unified `style.json`, defining precise layer styling for the extracted OSM primitives (highways, contours, waterways) and pointing strictly to the local `pmtiles://` and `/assets/` paths.

Phase 4: UI/UX Orchestration & Dynamic Theming (Week 6)

1. **Persistent WebGL Containerization:** Structure the React Router and component hierarchy to maintain the MapLibre `<canvas>` persistently in the DOM. Implement the CSS visibility toggle mechanism to transition between the Compilation HUD and the Interactive Map View without destroying the expensive GPU context.
2. **Dynamic Paint Engine:** Implement the theme mutation logic utilizing `map.setPaintProperty()` to enable instantaneous, shader-based switching between dark and light topographic palettes.
3. **Idempotent State Rehydration:** Finalize the application launch sequence. Implement checks that query the native layer for active or suspended `BGProcessingTask` or `WorkManager` compilation tasks upon startup, ensuring the UI progress bars instantly rehydrate to match the persistent Rust background state machine.

Works cited

1. On-Device Map Compilation - Feasibility, Architecture, and Implementation Plan.md
2. Serving a custom vector web map using PMTiles and maplibre-gl - Simon Willison: TIL, accessed July 14, 2026, <https://til.simonwillison.net/gis/pmtiles>
3. Bug Android serving local files net::ERR_FAILED - Stack Overflow, accessed July 14, 2026, <https://stackoverflow.com/questions/74220340/bug-android-serving-local-files-neterr-failed>
4. `node_modules/@capacitor/ios/Capacitor/Capacitor/CapacitorBridge.swift` · main · Anfri Hayward / Swen303 App Project - GitLab, accessed July 14, 2026, https://gitlab.ecs.vuw.ac.nz/haywarmatt/swen303-app-project/-/blob/main/node_modules/@capacitor/ios/Capacitor/Capacitor/CapacitorBridge.swift
5. bug: element playback issues on iOS using capacitor:// protocol #6790 - GitHub, accessed July 14, 2026, <https://github.com/ionic-team/capacitor/issues/6790>
6. 203302 - WKWebView: WKURLSchemeHandler request don't have Range headers - WebKit Bugzilla, accessed July 14, 2026, https://bugs.webkit.org/show_bug.cgi?id=203302
7. bug: iOS - playing video from filesystem / support range requests · Issue #5420 · ionic-team/capacitor - GitHub, accessed July 14, 2026,

- <https://github.com/ionic-team/capacitor/issues/5420>
8. bug: Android Range Requests fail for large files · Issue #7007 · ionic-team/capacitor - GitHub, accessed July 14, 2026, <https://github.com/ionic-team/capacitor/issues/7007>
 9. capgo/capacitor-fast-sql - GitHub, accessed July 14, 2026, <https://github.com/Cap-go/capacitor-fast-sql>
 10. Easily keep a backend database synced with in-app SQLite for offline-first/local-first Capacitor apps - Reddit, accessed July 14, 2026, https://www.reddit.com/r/capacitor/comments/1op1cna/easily_keep_a_backend_database_synced_with_inapp/
 11. Suggestion: document how to write binary files with Filesystem plugin · Issue #40 - GitHub, accessed July 14, 2026, <https://github.com/ionic-team/capacitor-file-system/issues/40>
 12. qrclip/capacitor-file-chunk - GitHub, accessed July 14, 2026, <https://github.com/qrclip/capacitor-file-chunk>
 13. Origin Private File System (OPFS) Database with the RxDB OPFS-RxStorage, accessed July 14, 2026, <https://rxdb.info/rx-storage-opfs.html>
 14. PMTiles/js/src/index.ts at main - GitHub, accessed July 14, 2026, <https://github.com/protomaps/PMTiles/blob/main/js/src/index.ts>
 15. FetchSource | pmtiles, accessed July 14, 2026, <https://pmtiles.io/typedoc/classes/FetchSource.html>
 16. Serve Protomaps Tiles with NodeJS - Craig Kochis, accessed July 14, 2026, <https://ckochis.com/serve-protomaps-tiles-with-nodejs>
 17. Protocol | pmtiles, accessed July 14, 2026, <https://pmtiles.io/typedoc/classes/Protocol.html>
 18. addProtocol() - MapLibre GL JS, accessed July 14, 2026, <https://maplibre.org/maplibre-gl-js/docs/API/functions/addProtocol/>
 19. React Native Talks, Workshops, Tutorials & Presentations: Your Guide to Mastering the JavaScript Framework - GitNation, accessed July 14, 2026, <https://gitnation.com/tags/react-native>
 20. 1 React Mouse Tracking Examples - FreeFrontend, accessed July 14, 2026, <https://freefrontend.com/react-mouse-tracking/>
 21. Best Practices for Optimizing Web Performance When Implementing Complex Animations in React - Zigpoll, accessed July 14, 2026, <https://www.zigpoll.com/content/can-you-explain-the-best-practices-for-optimizing-web-performance-when-implementing-complex-animations-in-react>
 22. Improve Web Performance With requestAnimationFrame - DebugBear, accessed July 14, 2026, <https://www.debugbear.com/blog/requestanimationframe>
 23. React pattern for very fast data fetching and rendering? : r/reactjs - Reddit, accessed July 14, 2026, https://www.reddit.com/r/reactjs/comments/m7klty/react_pattern_for_very_fast_data_fetching_and/
 24. Performance bottleneck when adding 8000+ PMTiles layers dynamically #5988 - GitHub, accessed July 14, 2026, <https://github.com/maplibre/maplibre-gl-js/discussions/5988>

25. Map - MapLibre GL JS, accessed July 14, 2026,
<https://maplibre.org/maplibre-gl-js/docs/API/classes/Map/>
26. Plugins - MapLibre GL JS, accessed July 14, 2026,
<https://maplibre.org/maplibre-gl-js/docs/plugins/>
27. How to display offline maps using Maplibre/Mapbox on Android | by Abhi - Medium, accessed July 14, 2026,
<https://medium.com/@ty2/how-to-display-offline-maps-using-maplibre-mapbox-39ad0f3c7543>
28. PMTiles for MapLibre GL - Protomaps Docs, accessed July 14, 2026,
<https://docs.protomaps.com/pmtiles/maplibre>
29. Add custom protocol PMTiles in the Web SDK - Microsoft Learn, accessed July 14, 2026,
<https://learn.microsoft.com/en-us/azure/azure-maps/add-custom-protocol-pmtiles>
30. PMTiles source and protocol - MapLibre GL JS, accessed July 14, 2026,
<https://maplibre.org/maplibre-gl-js/docs/examples/pmtiles-source-and-protocol/>
31. Vector Styles API - Thunderforest, accessed July 14, 2026,
<https://www.thunderforest.com/docs/vector-styles-api/>
32. muimsd/map-gl-offline: A TypeScript npm package for MapLibre GL JS and Mapbox GL JS to enable offline tiles. - GitHub, accessed July 14, 2026,
<https://github.com/muimsd/map-gl-offline/>
33. Self hosted maps for (practically) free - DEV Community, accessed July 14, 2026,
<https://dev.to/aaronblondeau/self-hosted-maps-for-practically-free-1i3n>
34. Proof-of-concept project using MapLibre with US PMTile and custom styles. - GitHub, accessed July 14, 2026,
<https://github.com/ryanblyth/maplibre-pmtiles-poc>
35. README.md - spatialillusions/maplibre-local-glyphs - GitHub, accessed July 14, 2026,
<https://github.com/spatialillusions/maplibre-local-glyphs/blob/main/README.md>